Test Case Specifications

Test Cases

Field	Detail
Test Case ID	**TC-001**
Test Description	Verify that a new patient can register with valid information.
Prerequisites	MongoDB database running, Backend server active, Email "john.doe@email.com" is not already registered.
Test Steps	1. Navigate to the registration page (/register). 2. Select "Patient" as the role. 3. Enter: Full Name="John Doe", Email="john.doe@email.com", Password="SecurePass@123". 4. Click the "Register" or "Submit" button.
Expected Result	Account is created successfully. Password is hashed in the database. User is redirected to the login page or their dashboard with a success message.
Actual Result	
Status	

Test Cases

Field	Detail
Test Case ID	**TC-002**
Test Description	Verify that a new doctor registration is processed correctly.
Prerequisites	MongoDB database running, Backend server active, Email "dr.smith@email.com" is not already registered.
Test Steps	1. Navigate to the registration page (/register). 2. Select "Doctor" as the role. 3. Enter: Full Name="Dr. Smith", Email="dr.smith@email.com", Password="SecurePass@123". 4. Click the "Register" button.
Expected Result	Account is created with role "doctor". Depending on the flow, the account may be created but require admin approval (is_active = false).
Actual Result	
Status	

Test Cases

Field	Detail
Test Case ID	**TC-003**
Test Description	Verify that registration fails when using an email that is already registered.
Prerequisites	A user with the email "existing@email.com" already exists in the database.
Test Steps	1. Navigate to the registration page. 2. Select any role. 3. Enter: Full Name="Jane Doe", Email="existing@email.com", Password="AnotherPass@123". 4. Click the "Register" button.
Expected Result	Registration fails with a 400 Bad Request status. An error message like "Email already registered" is displayed. No duplicate user record is created in the database.
Actual Result	
Status	

Test Cases

Field	Detail
Test Case ID	**TC-004**
Test Description	Verify that a user can log in successfully with valid credentials.
Prerequisites	A valid user account exists (e.g., email: "john@example.com", password: "Pass@123").
Test Steps	1. Navigate to the login page (/login). 2. Enter Email="john@example.com" and Password="Pass@123". 3. Click the "Login" button.
Expected Result	Login is successful. A 200 OK status is returned along with a valid JWT token. The user is redirected to their role-based dashboard.
Actual Result	
Status	

Test Cases

Field	Detail
Test Case ID	**TC-005**
Test Description	Verify that login fails with an incorrect password.
Prerequisites	A valid user account exists (e.g., email: "john@example.com", password: "CorrectPass@123").
Test Steps	1. Navigate to the login page. 2. Enter Email="john@example.com" and Password="WrongPass@456". 3. Click the "Login" button.
Expected Result	Login fails with a 401 Unauthorized status. A generic error message like "Invalid email or password" is displayed to the user.
Actual Result	
Status	

Test Cases

Field	Detail
Test Case ID	**TC-006**
Test Description	Verify that accessing a protected route without a JWT token is rejected.
Prerequisites	Backend server is running.
Test Steps	1. Using a tool like Postman or a browser, send a GET request to an authenticated endpoint, e.g., GET /api/v1/users/me. 2. Do not include an Authorization: Bearer <JWT_TOKEN> header.
Expected Result	The request is rejected with a 401 Unauthorized status.
Actual Result	
Status	

Test Cases

Field	Detail
Test Case ID	**TC-007**
Test Description	Verify that a patient cannot access an admin-only endpoint.
Prerequisites	A patient user is logged in and has a valid JWT token.
Test Steps	1. Using a tool like Postman, send a GET request to an admin endpoint, e.g., GET /api/v1/admin/reports/summary. 2. Include the patient's valid JWT token in the Authorization header.
Expected Result	The request is rejected with a 403 Forbidden status.
Actual Result	
Status	

Test Cases

Field	Detail
Test Case ID	**TC-008**
Test Description	Verify that a doctor cannot access an admin-only endpoint.
Prerequisites	A doctor user is logged in and has a valid JWT token.
Test Steps	1. Using a tool like Postman, send a POST request to an admin endpoint, e.g., POST /api/v1/admin/doctors. 2. Include the doctor's valid JWT token in the Authorization header.
Expected Result	The request is rejected with a 403 Forbidden status.
Actual Result	
Status	

Test Cases

Field	Detail
Test Case ID	**TC-009**
Test Description	Verify that an authenticated user can retrieve their own profile.
Prerequisites	A valid user (Patient, Doctor, or Admin) is logged in with a valid JWT token.
Test Steps	1. Send a GET request to an endpoint like /api/v1/auth/me or /api/v1/users/profile. 2. Include the user's valid JWT token in the Authorization header.
Expected Result	A 200 OK status is returned. The response body contains the user's details (e.g., id, name, email, role). The password hash must not be included in the response.
Actual Result	
Status	

Test Cases

Field	Detail
Test Case ID	**TC-010**
Test Description	Verify that an authenticated user can update their own profile information.
Prerequisites	A valid user is logged in. Current profile has name="Old Name", phone=null.
Test Steps	1. Send a PUT request to /api/v1/users/profile with a valid JWT token. 2. In the request body, provide updated fields: {"name": "New Name", "phone": "+94771234567"}. 3. Then, send a GET request to the same profile endpoint to verify the changes.
Expected Result	The PUT request returns a 200 OK status. The subsequent GET request returns the updated profile with "New Name" and the new phone number, confirming the database was updated.
Actual Result	
Status	

Test Cases

Field	Detail
Test Case ID	**TC-011**
Test Description	Verify that a patient can get a diabetes prediction with valid input data.
Prerequisites	Patient is logged in with a valid JWT token. ML models are loaded correctly on the server.
Test Steps	1. Send a POST request to /api/v1/predictions/diabetes with a valid JWT token. 2. Provide a valid request body, e.g., {"Pregnancies": 2, "Glucose": 120, "BloodPressure": 70, ...}.
Expected Result	A 200 OK status is returned. The response includes a prediction (0 or 1), risk_percentage, risk_level (e.g., Low/Medium/High/Critical), recommendation text, and video_recommendations.
Actual Result	
Status	

Test Cases

Field	Detail
Test Case ID	**TC-012**
Test Description	Verify that a patient can get a heart disease prediction with valid input data.
Prerequisites	Patient is logged in with a valid JWT token. ML models are loaded correctly.
Test Steps	1. Send a POST request to /api/v1/predictions/heart-disease with a valid JWT token. 2. Provide a valid request body, e.g., {"age": 52, "sex": 1, "cp": 0, "trestbps": 125, ...}.
Expected Result	A 200 OK status is returned. The response includes a prediction, risk_percentage, risk_level, and recommendation text.
Actual Result	
Status	

Test Cases

Field	Detail
Test Case ID	**TC-013**
Test Description	Verify that a patient can get a kidney disease prediction with valid input data.
Prerequisites	Patient is logged in with a valid JWT token. ML models are loaded correctly.
Test Steps	1. Send a POST request to /api/v1/predictions/kidney-disease with a valid JWT token. 2. Provide a valid request body, e.g., {"age": 48, "bp": 80, "sg": 1.02, "al": 1, ...}.
Expected Result	A 200 OK status is returned. The response includes a prediction, risk_percentage, risk_level, and recommendation text.
Actual Result	
Status	

Test Cases

Field	Detail
Test Case ID	**TC-014**
Test Description	Verify that a prediction request fails if a required field is missing.
Prerequisites	Patient is logged in with a valid JWT token.
Test Steps	1. Send a POST request to /api/v1/predictions/diabetes with a valid JWT token. 2. Provide a request body that is missing a required field, e.g., {"Pregnancies": 2, "BloodPressure": 70} (missing Glucose).
Expected Result	The request fails with a 422 Unprocessable Entity status. The response body contains a validation error message indicating which field is missing.
Actual Result	
Status	

Test Cases

Field	Detail
Test Case ID	**TC-015**
Test Description	Verify that a prediction request fails if a field has an invalid data type.
Prerequisites	Patient is logged in with a valid JWT token.
Test Steps	1. Send a POST request to /api/v1/predictions/diabetes with a valid JWT token. 2. Provide a request body with an invalid data type, e.g., {"Pregnancies": 2, "Glucose": "high", "BloodPressure": 70, ...}.
Expected Result	The request fails with a 422 Unprocessable Entity status. The response body contains a Pydantic validation error indicating the type mismatch.
Actual Result	
Status	

Test Cases

Field	Detail
Test Case ID	**TC-016**
Test Description	Verify that a successful prediction is saved to the patient's history.
Prerequisites	Patient is logged in and has just performed a successful prediction (e.g., from TC-011).
Test Steps	1. Send a GET request to /api/v1/predictions/history with the patient's valid JWT token.
Expected Result	A 200 OK status is returned. The response is a list that includes the most recently created prediction, containing the correct user_id, disease_type, and risk_level.
Actual Result	
Status	

Test Cases

Field	Detail
Test Case ID	**TC-017**
Test Description	Verify that a patient can filter their prediction history by disease type.
Prerequisites	Patient is logged in and has multiple predictions for different diseases (e.g., diabetes and heart disease).
Test Steps	1. Send a GET request to /api/v1/predictions/history?disease_type=diabetes with the patient's valid JWT token.
Expected Result	A 200 OK status is returned. The response list contains only predictions where the disease_type is "diabetes".
Actual Result	
Status	

Test Cases

Field	Detail
Test Case ID	**TC-018**
Test Description	Verify that a "High" or "Critical" risk prediction creates a notification for the patient.
Prerequisites	Patient is logged in. The system is configured to create notifications for high-risk results.
Test Steps	1. Perform a prediction (e.g., POST to /predictions/diabetes) with input data that is known to produce a "High" or "Critical" risk level. 2. Then, send a GET request to /api/v1/notifications with the patient's JWT token.
Expected Result	The first step returns a high-risk prediction. The second step returns a 200 OK with a list of notifications that includes a new, unread notification about the high-risk prediction result.
Actual Result	
Status	

Test Cases

Field	Detail
Test Case ID	**TC-019**
Test Description	Verify that the system uses a fallback recommendation if the LLM (Gemini) service fails.
Prerequisites	Patient is logged in. The server is configured such that the Gemini API call can be mocked to fail (e.g., network timeout, API error).
Test Steps	1. Trigger a prediction request (e.g., POST to /predictions/diabetes) with valid data. 2. The system attempts to call Gemini but fails (or is mocked to fail).
Expected Result	The prediction request still returns a 200 OK status. The recommendation field in the response contains a default message from FALLBACK_RECOMMENDATIONS. The server application does not crash.
Actual Result	
Status	

Test Cases

Field	Detail
Test Case ID	**TC-020**
Test Description	Verify that extreme input values are correctly mapped to "Critical" risk level.
Prerequisites	Patient is logged in.
Test Steps	1. Send a POST request to /api/v1/predictions/diabetes (or heart/kidney) with a valid JWT token. 2. Provide input values that are at the extreme end of the expected range (e.g., very high Glucose, BloodPressure, etc.).
Expected Result	A 200 OK status is returned. The risk_level field in the response is set to "High" or "Critical", as per the application's internal threshold logic.
Actual Result	
Status	

Test Cases

Field	Detail
Test Case ID	**TC-021**
Test Description	Verify that a doctor can retrieve a list of all patients.
Prerequisites	A doctor user is logged in with a valid JWT token. At least one patient exists in the database.
Test Steps	1. Send a GET request to /api/v1/users/patients with the doctor's valid JWT token.
Expected Result	A 200 OK status is returned. The response body contains a list of patient objects, each including at least id, name, and email.
Actual Result	
Status	

Test Cases

Field	Detail
Test Case ID	**TC-022**
Test Description	Verify that a doctor can view a specific patient's detailed profile.
Prerequisites	A doctor is logged in. A patient with patient_id "pt_001" exists and has some medical_history and allergies recorded.
Test Steps	1. Send a GET request to /api/v1/users/patients/pt_001/profile with the doctor's valid JWT token.
Expected Result	A 200 OK status is returned. The response body contains the patient's profile information, including the medical_history, allergies, and any existing doctor_remarks.
Actual Result	
Status	

Test Cases

Field	Detail
Test Case ID	**TC-023**
Test Description	Verify that a doctor can add a medical remark to a patient's profile.
Prerequisites	A doctor is logged in. A patient with patient_id "pt_001" exists.
Test Steps	1. Send a POST request to /api/v1/users/patients/pt_001/remarks with the doctor's JWT token. 2. Provide a request body: {"remark": "Patient shows signs of improvement."}. 3. Then, view the patient's profile again as per TC-022.
Expected Result	The POST request returns a 200 or 201 status. When viewing the profile (GET), the new remark is appended to the list of doctor_remarks.
Actual Result	
Status	

Test Cases

Field	Detail
Test Case ID	**TC-024**
Test Description	Verify that a doctor can list all predictions made by any patient.
Prerequisites	A doctor is logged in. Multiple predictions exist in the database from different patients.
Test Steps	1. Send a GET request to /api/v1/predictions/all with the doctor's valid JWT token.
Expected Result	A 200 OK status is returned. The response body contains a list of all predictions in the system.
Actual Result	
Status	

Test Cases

Field	Detail
Test Case ID	**TC-025**
Test Description	Verify that a doctor can add a review to a patient's prediction.
Prerequisites	A doctor is logged in. A prediction with prediction_id "pred_001" exists and does not yet have a doctor_review.
Test Steps	1. Send a POST request to /api/v1/predictions/pred_001/review with the doctor's JWT token. 2. Provide a request body: {"comment": "Prediction aligns with clinical assessment.", "notify_patient": true}.
Expected Result	A 200 OK status is returned. The prediction record is updated with the doctor_review. If notify_patient is true, a notification is created for the patient who owns the prediction.
Actual Result	
Status	

Test Cases

Field	Detail
Test Case ID	**TC-026**
Test Description	Verify that a doctor can create a new consultation for a patient.
Prerequisites	A doctor is logged in. A patient with patient_id "pt_001" exists.
Test Steps	1. Send a POST request to /api/v1/consultations with the doctor's JWT token. 2. Provide a request body: {"patient_id": "pt_001", "scheduled_at": "2024-05-20T10:00:00Z", "notes": "Follow-up on test results"}.
Expected Result	A 201 Created status is returned. A new consultation document is created in the database. The doctor_id in the new consultation is automatically set to the ID of the logged-in doctor from the JWT.
Actual Result	
Status	

Test Cases

Field	Detail
Test Case ID	**TC-027**
Test Description	Verify that a doctor can list only their own consultations.
Prerequisites	Doctor A is logged in. Consultations exist in the system for both Doctor A and Doctor B.
Test Steps	1. Send a GET request to /api/v1/consultations with Doctor A's valid JWT token.
Expected Result	A 200 OK status is returned. The response list contains only the consultations where the doctor_id matches Doctor A's ID.
Actual Result	
Status	

Test Cases

Field	Detail
Test Case ID	**TC-028**
Test Description	Verify that a doctor can update the status of their own consultation.
Prerequisites	Doctor A is logged in and owns a consultation with consultation_id "cons_001" that has status "scheduled".
Test Steps	1. Send a PATCH request to /api/v1/consultations/cons_001 with Doctor A's JWT token. 2. Provide a request body: {"status": "completed"}.
Expected Result	A 200 OK status is returned. The consultation record in the database is updated, and its status field is now "completed".
Actual Result	
Status	

Test Cases

Field	Detail
Test Case ID	**TC-029**
Test Description	Verify that a doctor cannot update another doctor's consultation.
Prerequisites	Doctor A is logged in. Consultation "cons_002" exists and is owned by Doctor B.
Test Steps	1. Send a PATCH request to /api/v1/consultations/cons_002 with Doctor A's JWT token. 2. Provide any update, e.g., {"status": "cancelled"}.
Expected Result	The request is rejected with a 403 Forbidden or a 404 Not Found status. The consultation owned by Doctor B remains unchanged.
Actual Result	
Status	

Test Cases

Field	Detail
Test Case ID	**TC-030**
Test Description	Verify that doctors receive notifications for high-risk patient predictions.
Prerequisites	A doctor is logged in. A high-risk prediction exists for a patient (or is triggered in this session).
Test Steps	1. Trigger a high-risk prediction for a patient (or ensure one exists). 2. Send a GET request to /api/v1/notifications with the doctor's JWT token.
Expected Result	A 200 OK status is returned. The notification list contains an entry about the high-risk patient, allowing the doctor to be aware of it.
Actual Result	
Status	

Test Cases

Field	Detail
Test Case ID	**TC-031**
Test Description	Verify that the admin dashboard summary report returns correct statistics.
Prerequisites	An admin user is logged in. The database contains known counts of patients, doctors, and predictions.
Test Steps	1. Send a GET request to /api/v1/admin/reports/summary with the admin's valid JWT token.
Expected Result	A 200 OK status is returned. The response body contains JSON with keys like total_patients, total_doctors, total_predictions, high_risk_count, and a breakdown by disease. These numbers should match the actual counts in the database.
Actual Result	
Status	

Test Cases

Field	Detail
Test Case ID	**TC-032**
Test Description	Verify that an admin can list all patients in the system.
Prerequisites	An admin user is logged in. Multiple patient records exist.
Test Steps	1. Send a GET request to the patient list endpoint (e.g., /api/v1/users/patients or an admin-specific equivalent) with the admin's JWT token.
Expected Result	A 200 OK status is returned. The response is a list containing all patient records, potentially including more details than a doctor would see.
Actual Result	
Status	

Test Cases

Field	Detail
Test Case ID	**TC-033**
Test Description	Verify that an admin can deactivate a user account.
Prerequisites	An admin is logged in. A user with user_id "usr_001" exists and is currently active (is_active: true).
Test Steps	1. Send a PATCH request to /api/v1/admin/users/usr_001 with the admin's JWT token. 2. Provide a request body: {"is_active": false}. 3. Attempt to log in as that user.
Expected Result	The PATCH request returns a 200 OK status. The user's is_active field in the database is set to false. The subsequent login attempt for that user fails with a 401 Unauthorized status.
Actual Result	
Status	

Test Cases

Field	Detail
Test Case ID	**TC-034**
Test Description	Verify that an admin can activate a deactivated user account.
Prerequisites	An admin is logged in. User "usr_001" is currently inactive (is_active: false).
Test Steps	1. Send a PATCH request to /api/v1/admin/users/usr_001 with the admin's JWT token. 2. Provide a request body: {"is_active": true}. 3. Attempt to log in as that user with their correct credentials.
Expected Result	The PATCH request returns a 200 OK status. The user's is_active field is set to true. The subsequent login attempt is successful.
Actual Result	
Status	

Test Cases

Field	Detail
Test Case ID	**TC-035**
Test Description	Verify that an admin can assign a doctor to a patient.
Prerequisites	An admin is logged in. A patient with patient_id "pt_001" and a doctor with doctor_id "dr_001" exist. The patient does not currently have an assigned doctor.
Test Steps	1. Send a POST request to /api/v1/admin/patients/pt_001/assign-doctor with the admin's JWT token. 2. Provide a request body: {"doctor_id": "dr_001"}.
Expected Result	A 200 OK status is returned. When viewing the patient's profile, the assigned_doctor_id field is now set to "dr_001".
Actual Result	
Status	

Test Cases

Field	Detail
Test Case ID	**TC-036**
Test Description	Verify that an admin can unassign a doctor from a patient.
Prerequisites	An admin is logged in. Patient "pt_001" currently has an assigned doctor "dr_001".
Test Steps	1. Send a DELETE request to /api/v1/admin/patients/pt_001/assign-doctor with the admin's JWT token.
Expected Result	A 200 OK status is returned. When viewing the patient's profile, the assigned_doctor_id field is now null or cleared.
Actual Result	
Status	

Test Cases

Field	Detail
Test Case ID	**TC-037**
Test Description	Verify that an admin can create a new doctor account.
Prerequisites	An admin is logged in. Email "new.doctor@email.com" is not already in use.
Test Steps	1. Send a POST request to /api/v1/admin/doctors with the admin's JWT token. 2. Provide a request body: {"name": "New Doctor", "email": "new.doctor@email.com", "password": "TempPass@123"}.
Expected Result	A 201 Created status is returned. A new user with the role "doctor" is created in the database.
Actual Result	
Status	

Test Cases

Field	Detail
Test Case ID	**TC-038**
Test Description	Verify that an admin can delete (or deactivate) a doctor account.
Prerequisites	An admin is logged in. A doctor with doctor_id "dr_002" exists.
Test Steps	1. Send a DELETE request to /api/v1/admin/doctors/dr_002 with the admin's JWT token.
Expected Result	A 200 OK or 204 No Content status is returned. The doctor user is either removed from the database or their is_active status is set to false.
Actual Result	
Status	

Test Cases

Field	Detail
Test Case ID	**TC-039**
Test Description	Verify that an admin can retrieve the current system settings.
Prerequisites	An admin is logged in. System settings are configured in the database.
Test Steps	1. Send a GET request to /api/v1/admin/settings with the admin's valid JWT token.
Expected Result	A 200 OK status is returned. The response body contains the system settings, such as systemName, maxFileSize, sessionTimeout, etc.
Actual Result	
Status	

Test Cases

Field	Detail
Test Case ID	**TC-040**
Test Description	Verify that an admin can update and save system settings.
Prerequisites	An admin is logged in.
Test Steps	1. Send a PUT request to /api/v1/admin/settings with the admin's JWT token. 2. Provide an updated settings object, e.g., {"systemName": "New Health System Name", "maxFileSize": 20, ...}. 3. Send a GET request to the settings endpoint again to verify.
Expected Result	The PUT request returns a 200 OK status. The subsequent GET request returns the new, updated settings values, confirming they were saved to the database.
Actual Result	
Status	

Test Cases

Field	Detail
Test Case ID	**TC-041**
Test Description	Verify that an admin can list the available ML models.
Prerequisites	An admin is logged in. Model files (e.g., diabetes_model.pkl) and threshold files are present on the server.
Test Steps	1. Send a GET request to /api/v1/admin/models with the admin's valid JWT token.
Expected Result	A 200 OK status is returned. The response body contains a list of model files and associated configuration files present in the system.
Actual Result	
Status	

Test Cases

Field	Detail
Test Case ID	**TC-042**
Test Description	Verify that an admin can view all consultations in the system.
Prerequisites	An admin is logged in. Consultations exist for multiple different doctors.
Test Steps	1. Send a GET request to /api/v1/consultations with the admin's valid JWT token.
Expected Result	A 200 OK status is returned. The response body contains a list of all consultations, regardless of which doctor owns them.
Actual Result	
Status	

Test Cases

Field	Detail
Test Case ID	**TC-043**
Test Description	Verify that a patient can only see their own notifications.
Prerequisites	Patient A is logged in. There are notifications in the system for Patient A and for Patient B.
Test Steps	1. Send a GET request to /api/v1/notifications/ with Patient A's valid JWT token.
Expected Result	A 200 OK status is returned. The response list contains only the notifications where the user_id matches Patient A's ID. Notifications for Patient B are not present.
Actual Result	
Status	

Test Cases

Field	Detail
Test Case ID	**TC-044**
Test Description	Verify that a user can mark a notification as read.
Prerequisites	A user (Patient or Doctor) is logged in and has at least one unread notification with notification_id "notif_001".
Test Steps	1. Send a PUT request to /api/v1/notifications/notif_001/read with the user's valid JWT token. 2. Fetch the list of notifications again.
Expected Result	The PUT request returns a 200 OK status. In the subsequent GET request, the is_read field for "notif_001" is now true.
Actual Result	
Status	

Test Cases

Field	Detail
Test Case ID	**TC-045**
Test Description	Verify that the chatbot returns a valid AI response for a valid input message.
Prerequisites	Any logged-in user (Patient, Doctor, Admin). The AI chatbot service (e.g., Gemini) is available.
Test Steps	1. Send a POST request to /api/v1/chatbot/message with the user's valid JWT token. 2. Provide a request body: {"message": "What is a healthy blood pressure range?"}.
Expected Result	A 200 OK status is returned. The response body contains a relevant and coherent AI-generated response to the query.
Actual Result	
Status	

Test Cases

Field	Detail
Test Case ID	**TC-046**
Test Description	Verify that the chatbot rejects an empty or very short message.
Prerequisites	Any logged-in user is logged in.
Test Steps	1. Send a POST request to /api/v1/chatbot/message with the user's valid JWT token. 2. Provide a request body with an empty message: {"message": ""}.
Expected Result	The request is rejected with a 400 Bad Request or 422 Unprocessable Entity status, indicating the message is too short or invalid.
Actual Result	
Status	

Test Cases

Field	Detail
Test Case ID	**TC-047**
Test Description	Verify that a patient record can be stored and then retrieved.
Prerequisites	A user with permission to store records (e.g., Doctor/Admin) is logged in.
Test Steps	1. Send a POST request to a record storage endpoint (e.g., /api/v1/patient-records/store) with a valid JWT token and a file/record. 2. Send a GET request to retrieve the patient's records (e.g., /api/v1/patient-records?patient_id=pt_001).
Expected Result	The first step returns a success status (e.g., 201). The second step returns a 200 OK with a list of records, including the one just stored, correctly associated with patient_id.
Actual Result	
Status	

Test Cases

Field	Detail
Test Case ID	**TC-048**
Test Description	Verify that the system handles a missing ML model file gracefully.
Prerequisites	The ML model file for a specific disease (e.g., diabetes_model.pkl) is missing from its expected path, or the path in the configuration is incorrect.
Test Steps	1. Log in as a patient. 2. Send a POST request to the prediction endpoint for that disease (e.g., /api/v1/predictions/diabetes) with valid input data.
Expected Result	The request does not cause an unhandled server crash. A 500 Internal Server Error or 503 Service Unavailable status is returned with a clear error message indicating the model file could not be loaded.
Actual Result	
Status	

Test Cases

Field	Detail
Test Case ID	**TC-049**
Test Description	Verify that an inactive user cannot log in.
Prerequisites	A user account exists with is_active: false (e.g., a deactivated patient or a doctor pending approval). The user's credentials are known.
Test Steps	1. Navigate to the login page. 2. Enter the email and password of the inactive user. 3. Click the "Login" button.
Expected Result	Login fails with a 401 Unauthorized status. A generic error message (like "Invalid email or password") may be displayed, but not one that reveals the account is inactive for security reasons.
Actual Result	
Status	

Test Cases

Field	Detail
Test Case ID	**TC-050**
Test Description	Verify that a patient can only view their own consultations.
Prerequisites	Patient A is logged in. Consultations exist in the system for Patient A and for Patient B.
Test Steps	1. Send a GET request to /api/v1/consultations with Patient A's valid JWT token.
Expected Result	A 200 OK status is returned. The response list contains only the consultations where the patient_id matches Patient A's ID. Consultations for Patient B are not present.
Actual Result	
Status	